ELSEVIER

17 July 1995

PHYSICS LETTERS A

Physics Letters A 203 (1995) 107–114

Creating periodic orbits in chaotic systems

Kathryn Glass, Michael Renton, Kevin Judd, Alistair Mees

Centre of Applied Dynamics and Optimization, Department of Mathematics, University of Western Australia, Nedlands 6907, Australia

Received 6 February 1995; revised manuscript received 24 April 1995; accepted for publication 24 May 1995
Communicated by A.P. Fordy

Abstract

We introduce methods of forcing many types of behavior from a chaotic map using only variations of simple targeting algorithms. In particular, we show how to create new periodic orbits which are close to existing non-periodic orbits. We also look at robust targeting algorithms that will target and stabilize chaotic maps in the presence of noise.

1. Introduction

In this Letter, which summarizes ideas presented in Ref. [1], we look at controlling chaotic maps using small perturbations. We discuss the application of basic targeting algorithms to systems containing noise or modeling errors. We also look at methods of controlling a chaotic system to display various behaviors.

One of the features of a chaotic system is that small initial perturbations can produce large changes in the system in the long term. We can make use of this property to control systems by making certain small perturbations to the system which will produce the long term changes we require. We will discuss algorithms for controlling chaotic maps of the form

$$x_{n+1} = f(x_n, p_n),$$

where x_n is the state of the system at the nth iteration and p_n is the value of some control parameter at the nth iteration. Although many of the results in this Letter may be extended to more general systems, we will confine our attention to two-dimensional maps with a one-dimensional unstable manifold. We also assume that the system has an attractor, although the results may be extended to conservative systems with a one-dimensional unstable manifold.

In general, the system has an unperturbed value, $\hat{p}$, and we define a set P of allowable perturbations, where P is a some neighborhood of $\hat{p}$. We then make the following definition:

Definition 1. A sequence $y = \{y_t\}_{t=1}^{M}$ is a *P-chain* if, for each $t = 1, 2, \ldots, M-1$, y_{t+1} is in the set $f(y_t, P) = \{f(y_t, p) : p \in P\}$.

In other words, a P-chain is a trajectory that may result when perturbations from the set P are made to the state of the system at each iteration. We can extend this notion to consider P-chains with recurrence properties. We define a *periodic P-chain* as follows:

Definition 2. A sequence $y = \{y_t\}_{t=1}^{M}$ is a *periodic P-chain* if y is a P-chain and $y_1 \in f(y_M, P)$.

So, a periodic P-chain is a P-chain where perturbations can be chosen so that the trajectory repeats itself after every M iterations. Although in general it is not a periodic orbit of the original unperturbed system, it behaves like an orbit. In order to look at creating and

SSDI 0375-9601(95)00398-3

then targeting these periodic P-chains, we need to review some basic targeting algorithms.

2. Targeting algorithms

The theory behind targeting sets and points in chaotic systems has been well developed in Refs. [2–5]. We will review some of these targeting problems and then discuss algorithms that will solve them. For an extensive bibliography, see Ref. [6]. We wish to find an algorithm that will allow us to target a neighborhood of a certain point in state space from some other point in state space. In other words, given a starting point x_s and a target set T we want to find a P-chain $y = \{y_t\}_{t=1}^{M}$ of length M with initial point $y_1 = x_s$ and final point y_M in T.

In order to derive algorithms for solving this problem, we need to consider the sets that may be reached from our starting point x_s. If we recall that the equation governing the motion is given by $x_{n+1} = f(x_n, p_n)$, we can define the set S_1 of points that may be reached from x_s under one iterate by

$$S_1 = \{f(x_s, p) : p \in P\}.$$

Similarly, we can define reachable sets S_n for $n > 1$ by

$$S_n = \{f(x, p) : x \in S_{n-1} \text{ and } p \in P\}.$$

In order for the reachable sets to have the properties we require for controlling purposes, we need to be able to control in a direction transverse to the unstable manifold of the system. In the two-dimensional map we are considering, one parameter will be sufficient to ensure this in nearly all cases. The repeated iteration of the function f will cause the reachable sets to be compressed onto, and stretched along, the attractor of the unperturbed system. In fact, we find that if we take n large enough, the reachable set S_n will contain points arbitrarily close to any point on this attractor (see Refs. [7,4]). Thus, if the target set T intersects the attractor of the unperturbed system, there should be a solution for any perturbation set P containing $\hat{p}$. In particular, we can consider a special case of this problem where we choose any initial perturbation p_1 in P, but thereafter restrict the p_i to be $\hat{p}$. When we consider the reachable sets, the set S_1 is unchanged and the set S_n for $n = 2, \ldots, N$ is given by

$$S_n = \{f^{n-1}(f(x_s, p), \hat{p}) : p \in P\}.$$

As before, if T intersects the attractor of the unperturbed system, this problem has a solution. In the following when we refer to the attractor of the system we mean the attractor of the unperturbed system.

We may now consider an algorithm, which we will refer to as the initial perturbation algorithm which solves the targeting problem in the case where the target set intersects the attractor.

Initial perturbation algorithm.
- We are given a starting point x_0 and a target set T.
  Set n to 1.
- **while** (S_n does not intersect T) **do**
  - Evaluate S_n.
  - Increment n.
- Choose x^* in the intersection of S_n and T.
  Set p^* to be the corresponding initial perturbation.

The initial perturbation algorithm will give us an initial perturbation p^* that will target the trajectory into the set T under no further perturbations to the system, provided there is no noise in the system. If the system does contain noise, further perturbations will be necessary; this case is discussed in Section 3.

Clearly, when we come to implement this, we will need to approximate the reachable sets by finite collections of points. Every point in each reachable set is stored together with the initial perturbation that corresponds to it. Once such a set intersects the target set, choosing a point in this intersection automatically produces the initial perturbation that targets the set. The accuracy of the implementation will depend on the number of points that we use in the approximation of the reachable sets. We may also consider extensions of the algorithm that refine the targeting perturbation.

The initial perturbation algorithm is very effective at targeting points lying on the attractor. When we require a targeting algorithm to reach a fixed point or periodic orbit prior to stabilizing it, this algorithm will be quite adequate. In some cases, however, we may wish the system to enter some region of phase space that is not on the attractor of the unperturbed system. In this case, it is unlikely that any of the reachable sets defined by the initial perturbation algorithm will intersect the target set, as they are continually pushed closer to the attractor. However, we can choose a fur-

ther perturbation to move the trajectory off the attractor. Clearly, we want to make this second perturbation as late as possible, as the trajectory will be pushed back onto the attractor with each further iteration. We thus consider a more complex algorithm in which the value of p_i is perturbed at the first and at the last iteration of the function.

In order to take into account the final perturbation, we define the set T_1 to be those points that may be perturbed into the set T under one iteration of the function. In other words,

$$T_1 = \{x : f(x,p) \in T \text{ and } p \in P\}.$$

We consider a new algorithm that applies the initial perturbation algorithm with target set T_1. This produces an initial perturbation p_I that targets the trajectory to a point $\hat{x}$ in the set T_1. The algorithm then sets p_F to be the perturbation that will target $\hat{x}$ into T. The algorithm outputs the initial perturbation p_I and the final perturbation p_F that will direct the trajectory to the set T. We will refer to this algorithm as the initial and final perturbation algorithm.

In general, this algorithm allows us to target a point more exactly than the initial perturbation algorithm, particularly if the point lies off the attractor. The magnitude of the maximum allowed perturbation determines the distance from the attractor that can be targeted. Clearly, if this is very small, the initial and final perturbation algorithm will not perform significantly better than the initial perturbation algorithm.

In the algorithms outlined above, we have assumed that the system is noiseless. We encounter problems when applying these algorithms to systems in which there is noise or modeling errors. In the next section, we look at extensions to these algorithms that compensate for noise.

3. Compensating for noise

It is clear that any noise or errors that occur in modeling will cause a system to wander from a calculated targeting P-chain. In many cases, the resulting trajectory will fail to reach the target set. The standard method of compensating for this is to re-apply the targeting algorithm after each iteration (or at some determined frequency) in order to ensure that the trajectory hits the target (see Ref. [4]) The disadvantage of this method is that each P-chain that is calculated is discarded after one iteration, and a new chain determined. Clearly, for a long trajectory, this is very time consuming. We outline an alternative method that will calculate a single P-chain to hit our target and then allow us to follow this trajectory thereafter.

Clearly we can modify either of our targeting algorithms to output the chain associated with the targeting perturbations. Rather than discard the chain at each iteration of the function and find a new one, we look at applying an algorithm that will allow us to track, or follow a P-chain. By making small perturbations at each iteration, it is possible to compensate for noise or modeling errors that disturb the trajectory from the P-chain. We do this by simply applying a form of the targeting algorithm to target a small neighborhood of a point in the P-chain a few iterates ahead on the trajectory. We will refer to this tracking algorithm as the correcting algorithm.

Correcting algorithm.

- We are given a P-chain $y = \{y_t\}_{t=i}^{M}$ with $M \geqslant i+2$ such that the current state of the system, x_n, is in the neighborhood of y_i.
- Apply the initial perturbation algorithm with $x_s = x_n$, a specified number of steps $N = 2$ and a target set, T, some small neighborhood of y_{i+2}.
  The algorithm produces the optimal initial perturbation p^*.
- Calculate $x_{n+1} = f(x_n, p^*)$, the new state of the system.

Applying the initial perturbation algorithm to target a set in two steps can be done very efficiently, as the reachable sets may be approximated by a small number of points. Thus the correcting algorithm will produce a correcting perturbation very quickly and provide an efficient way of tracking a P-chain. We can now define a targeting algorithm that will find a P-chain to target some set and then follow this chain using the correcting algorithm. We will refer to this algorithm as the targeting algorithm.

Targeting algorithm.

- We are given a starting point $x_s = x_0$ and a target set T.
- Apply the initial and final perturbation algorithm to find a P-chain $y = \{y_t\}_{t=0}^{M}$ with initial perturbation

p_1 such that $y_0 = x_s$ and y_M is in the set T.
Set $p_{n-1} = p_1$, set $i = 1$ and set $n = 1$.
- **while** $i < M$ **do**
  - Calculate $x_n = f(x_{n-1}, p_{n-1})$.
  - **if** $M > i + 2$
    - Apply the correcting algorithm to the P-chain $y = \{y_t\}_{t=i}^{M}$
    - This produces a perturbation p^*.
  - **else if** $M = i + 2$
    - Apply the initial and final perturbation algorithm with starting point x_n and target set T.
    - Let p^* be the initial perturbation produced by this algorithm.
  - **else**
    - Apply the initial perturbation algorithm with starting point x_n and target set T.
    - This produces a perturbation of p^*.
  - Set $p_n = p^*$.
  - Increment n. Increment i.
  - **end while**
- Put $x_n = f(x_{n-1}, p_{n-1})$. Then x_n is in T.

The above algorithm first determines a path, or P-chain, targeting the set T in M iterations. It then uses the correcting algorithm to stay on that path until it is within two steps of the target. The target set itself is then targeted at the last two iterations using firstly the initial and final perturbation algorithm and then the initial perturbation algorithm.

When implementing this algorithm we must take some care to ensure that we allow the correcting algorithm sufficient scope to follow the P-chain. If the P-chain itself includes some relatively large perturbation at any step, we may find that the extra perturbation required by the correcting algorithm to follow the chain at this step causes the total perturbation to leave the set P of allowable perturbations. One way of solving this problem is to make further restrictions on the perturbations allowed when creating the P-chain in the first place. In other words, we define some suitably small proper subset P' of P containing the unperturbed value $\hat{p}$. The first step in the targeting algorithm is then to find a P'-chain that targets the set T. When it comes to applying the correcting algorithm to follow this chain, we allow ourselves perturbations from the set P. We must also ensure that the perturbations are sufficient to overcome the noise in the system. Once the noise level reaches the order of the allowable perturbations, we may be unable to compensate for it by adjusting the control parameter.

In order to test these algorithms, we consider the following system, which is similar to, but more complex than, the Hénon system,

$$\begin{pmatrix} x_{n+1} \\ y_{n+1} \end{pmatrix} = f \begin{pmatrix} x_n \\ y_n \end{pmatrix} = \begin{pmatrix} p + 0.3y_n - 0.7x_n^3 + 2.1x_n \\ x_n \end{pmatrix}. \quad (1)$$

The control parameter in this system is p and the unperturbed value, $\hat{p}$, is zero.

Let us consider some examples using the system defined by (1). In Fig. 1 we present two examples in which points are targeted in a system with a noise term included. The noise is uniformly distributed with magnitude less than or equal to 0.005, the starting point is (1, 1) in both cases and the maximum perturbation allowed in the algorithms is 0.01 (that is, $P = (-0.01, 0.01)$). In the first example, the algorithm is applied for 18 steps and in the second it is applied for 13 steps. The target point is represented by a circle and the point reached by the targeting algorithm by an asterisk. The point that is reached if the initial and final perturbations calculated are applied to the system with no corrections made for the noise is represented by a plus. An approximation of the attractor is plotted in the background. Clearly, without the corrections made to the system, the trajectory wanders far from the target.

There are further applications of the correcting algorithm, other than to compensate for noise when targeting a certain set. When we try to stabilize high order periodic orbits, the stabilization procedures that carry through from control theory (discussed in Refs. [8,9,3]) turn out to be highly sensitive to noise and useful only on a small neighborhood of the orbit (see Ref. [7]). Since a periodic orbit is, in effect, a special case of a P-chain, we can use the correcting algorithm to stabilize the orbit. Once we are in some neighborhood of the orbit, continually targeting a point a few iterates further on will cause us to remain in the neighborhood of the orbit. We will refer to this algorithm as the P-chain stabilization algorithm.

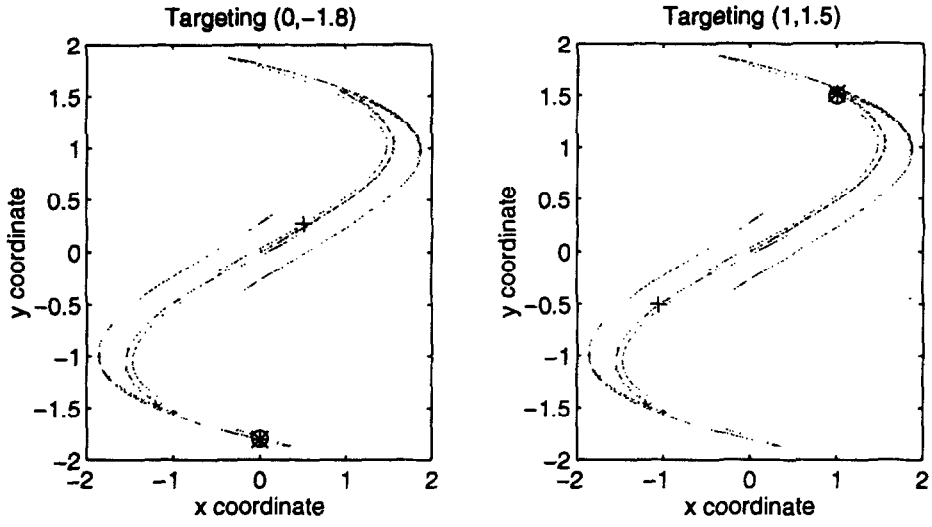

Fig. 1. Two examples of points targeted from starting point (1, 1) by the targeting algorithm. The target point is marked with a circle, the point reached by the targeting algorithm is marked by an asterisk and the point reached if no correction is made for the noise is marked with a plus. An approximation of the attractor is plotted in the background.

P-chain stabilization algorithm.

- We are given a P-chain $y = \{y_t\}_{t=1}^{M}$. The current state of the system, x_n is in the neighborhood of y_1.
- **for** $i = 1$ **to** $M - 2$ **do**
  - Apply the correcting algorithm to target y_{i+2}.
  - The algorithm produces the optimal initial perturbation p^*.
  - Calculate $x_{n+1} = f(x_n, p^*)$, the new state of the system.
  - Set $n = n + 1$.

Although this algorithm has been designed to stabilize high-order periodic orbits, it is also relevant to orbits of any period, including fixed points. If we combine this algorithm with the targeting algorithm we may target and stabilize such orbits in a system containing noise. To give an example, we again consider the system defined by Eq. (1) including a noise term. The noise is uniformly distributed with magnitude less than or equal to 0.005. We apply the targeting algorithm and the P-chain stabilization algorithm to target and then stabilize periodic points in this system. We restrict the allowable perturbations in these algorithms to have magnitude less than or equal to 0.015. Fig. 2 provides a graph of the x coordinate of the system at each iteration as two period two-points and a period one-point are targeted in turn. We represent the points

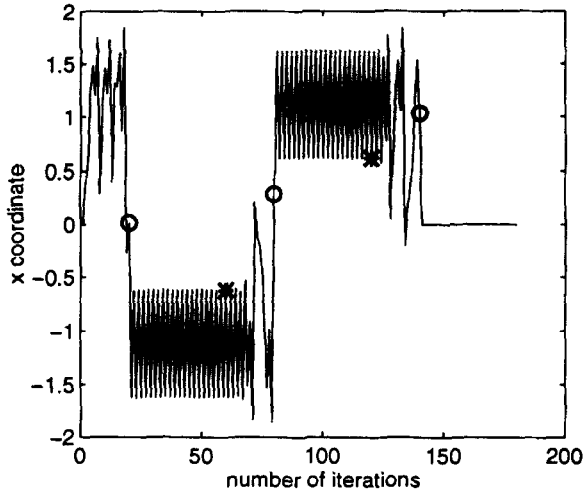

Fig. 2. Graph showing the x coordinate of the state vector as the targeting algorithm and the P-chain stabilization algorithm are applied to the system given by Eq. (1) containing a noise term. The points at which the targeting algorithm is applied are indicated by circles and the points at which the stabilization algorithm is discontinued are indicated by asterisks.

at which the targeting algorithm is implemented by circles and the points at which the stabilization algorithm is discontinued by asterisks.

Clearly the algorithms discussed in this section are

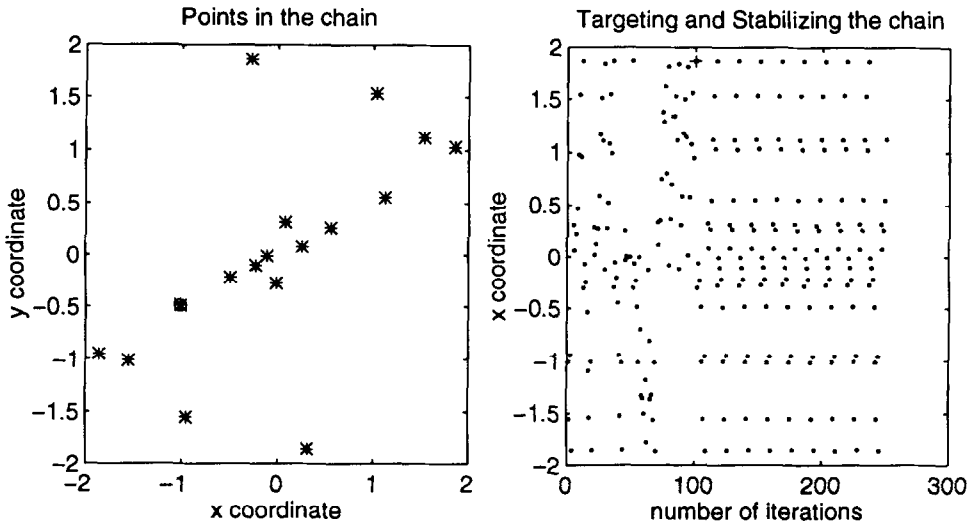

Fig. 3. The first graph shows 17 points in a periodic P-chain with the initial point marked with a circle. The second graph shows the targeting and stabilizing of this chain. The system is allowed to wander unperturbed initially, and the point at which the targeting algorithm is applied is indicated with a plus.

effective for targeting and stabilizing points in a system containing noise. We note that the P-chain stabilization algorithm is defined in terms of P-chains, although so far we have applied it only to the special case of a periodic orbit. We will find that the algorithm is equally useful for P-chains with non-trivial perturbations.

4. Creating periodic P-chains

We have already discussed one of the main problems in controlling chaotic systems – that of targeting certain points or sets. Another problem that is often encountered is that of placing some restrictions on the behavior of the system. For example, we may require the system to avoid certain areas of state space, or for it to visit the neighborhood of a certain point periodically. The standard approach to solving this problem is to locate a periodic orbit in the system that displays the characteristics we require and then attempt to stabilize this orbit (see Ref. [10]). Finding such a periodic orbit can be extremely time consuming, especially in high dimensional or complex systems. In addition to this, we cannot expect to locate periodic orbits exactly in a real-world system, and have to make approximations. This leads us to the idea of creating periodic orbits of our own.

By applying the targeting algorithms previously discussed, it is possible to start at some initial point x_s and then target back to this point in a certain number of iterations (the period). Clearly the resulting trajectory is not a periodic orbit of the original system, as there are perturbations that allow it to return to itself. It is, however, a *periodic P-chain.*

Mostly, when we choose to invent our own periodic P-chain, it will have a fairly high period. As we have already discussed, in this situation it is preferable to use the P-chain stabilization algorithm of the previous section rather than stabilization algorithms using linearized feedback control. The P-chain stabilization algorithm is applicable to any P-chain, and need not be restricted to the special case of an unperturbed periodic orbit.

Let us now look at an example using system (1). Fig. 3 presents two plots. The first shows the points in a periodic P-chain and the second shows the targeting and stabilizing algorithms applied to this chain. We start at the point $(-1, -0.5)$ and target this point with a maximum allowable perturbation of 0.005. This produces a chain of period 17, the points of which are graphed with the initial point indicated with a circle. We then stabilize this chain in the system containing a noise term of magnitude less than or equal

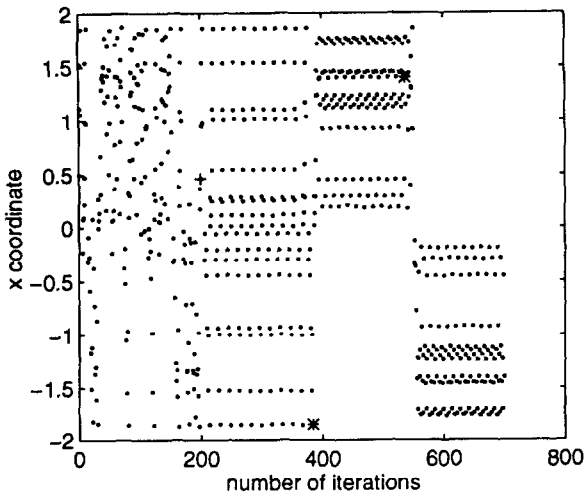

Fig. 4. The x coordinate at each iteration as three periodic P-chains are stabilized in turn. The targeting algorithm is started at the point marked with a plus and the asterisks indicate where the P-chain stabilization algorithm switches from one periodic P-chain to the next.

to 0.005 with perturbations to the system of magnitude at most 0.015. Initially, we start the system at the point $(-1, -0.5)$ and allow it to wander unperturbed for 100 iterations. It is clear from the graph that the behavior is not periodic when unperturbed. In other words, the periodic P-chain is not a periodic orbit, and has indeed been "created". At the point indicated with a plus, the targeting algorithm is used to target a point in the chain. The P-chain stabilization algorithm is then implemented and the P-chain is stabilized for over 100 iterations.

The ideas behind the creation of periodic P-chains may be extended even further. If there are a variety of different behaviors that we require for our system, we may wish to locate a number of periodic P-chains displaying these required behaviors. We can then find further P-chains that allow us to move between these different periodic P-chains. Once we have built up a library of these P-chains we will be able to control the system to display any behavior, and switch from one behavior to another at any time. This can be done very simply using only the P-chain stabilization algorithm.

We can demonstrate these ideas using system (1). First we locate three periodic P-chains in this system each displaying different behaviors (see Fig. 4). The first has points distributed throughout phase space, the second chain consists only of points in the positive quadrant of phase space, and the third chain consists of points in the negative quadrant of phase space. All these chains have perturbations of maximum magnitude 0.005. We use the targeting algorithm to find P-chains connecting these periodic P-chains, again restricting the perturbations to a maximum magnitude of 0.005. These chains are then stabilized in the system with a noise term of magnitude less than or equal to 0.005, with perturbations of magnitude at most 0.015. A plot of the x coordinate at each iteration is given in Fig. 4.

Firstly we allow the system to wander chaotically for 200 iterations. We then implement the targeting algorithm to target a point on the first periodic P-chain. The point at which this algorithm is implemented is indicated with a plus. Once we reach the first periodic P-chain, we stabilize it using the P-chain stabilization algorithm. After stabilizing the chain for ten periods, we then locate the P-chain in our library that connects the first periodic P-chain to the second. We can track this chain using the P-chain stabilization algorithm, and then stabilize the second periodic P-chain as before. The point at which the system begins to track the connecting P-chain is indicated in Fig. 4 by an asterisk. We then repeat this procedure to move from the second to the third chain.

In the example given above, it is perhaps unnecessary to find chains connecting the periodic P-chains before we begin to stabilize them. We have only used the chains once, and it would be just as quick to use the targeting algorithm to switch between them. However, in a real-world situation, we are likely to want to move frequently between a number of chains, and it is then advantageous to have the necessary trajectories stored in advance. A possible application of this is in the use of chaos in communications, as discussed in Ref. [11], where frequent switching between orbits is required. Once we are in the neighborhood of any of our calculated chains, we require only the P-chain stabilization algorithm to direct the system to behave in any way we require.

Acknowledgement

The authors wish to acknowledge the suggestions of an anonymous referee on the extension of the re-

sults to conservative systems. This research is partially supported by the A.R.C.

References

[1] M. Renton, Honours Dissertation. Controlling chaos with smart butterflies, unpublished (1994).
[2] E.J. Kostelich, C. Grebogi, E. Ott and J.A. Yorke, Phys. Rev. E 47 (1993) 305.
[3] T. Shinbrot, C. Grebogi, E. Ott and J.A. Yorke, Nature 363 (1993) 411.
[4] T. Shinbrot, E. Ott, C. Grebogi and J.A. Yorke, Phys. Rev. Lett. 65 (1990) 3215.
[5] T. Shinbrot, E. Ott, C. Grebogi and J.A. Yorke, Phys. Rev. Lett. 68 (1992) 2863.
[6] G. Chen, Control and synchronization of chaotic systems, EE Department, University of Houston, TX, USA, available from ftp: uhoop.egr.uh.edu/pub/TeX/chaos.tex (login name and password: both "anonymous").
[7] T. Shinbrot, E. Ott, C. Grebogi and J.A. Yorke, Phys. Rev. A 45 (1992) 4165.
[8] D. Auerbach, C. Grebogi, E. Ott and J.A. Yorke, Phys. Rev. Lett. 69 (1992) 3479.
[9] F.J. Romeiras, C. Grebogi, E. Ott and W.P. Dayawansa, Physica D 58 (1992) 165.
[10] E. Ott, C. Grebogi and J.A. Yorke, Phys. Rev. Lett. 64 (1990) 1196.
[11] S. Hayes, C. Grebogi and E. Ott, Phys. Rev. Lett. 70 (1993) 3031.